

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И ИССЛЕДОВАНИЙ
РЕСПУБЛИКИ МОЛДОВА
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ МОЛДОВЫ
Факультет вычислительной техники, информатики и микроэлектроники
Кафедра «Компьютерных наук и системной инженерии»**

ОТЧЁТ

Лабораторная работа № 1

по дисциплине «Компьютерное программирование»

Тема: Использование операторов проверки условия
и операторов цикла на языке Си

Вариант 7

Выполнил: студент гр. CR-263
Строков Арсений

Проверил: Н. С. Фалько

КИШИНЁВ 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ЦЕЛЬ И ХОД РАБОТЫ.....	3
1.1 Цель работы.....	3
1.2 Ход работы.....	3
2 ЗАДАНИЕ.....	4
2.1 Условия выбора ветвей и область определения.....	4
2.2 Поразрядное условие.....	4
3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ.....	5
3.1 Код программы, написанной самостоятельно.....	5
3.2 Блок-схема алгоритма.....	7
3.3 Код программы, сгенерированный DeepSeek.....	8
4 СРАВНЕНИЕ ДВУХ ПРОГРАММ.....	10
4.1 Сравнительная таблица.....	10
4.2 Результаты работы программ.....	12
ЗАКЛЮЧЕНИЕ.....	16
ПРИЛОЖЕНИЕ. ССЫЛКА НА РЕПОЗИТОРИЙ.....	17

ВВЕДЕНИЕ

Операторы проверки условия (if, else if, else) и операторы цикла (for, while, do while) относятся к основным конструкциям языка Си. Первые позволяют программе выполнять разные действия в зависимости от исходных данных, вторые — многократно повторять однотипные вычисления.

Типовая задача, на которой удобно отрабатывать эти конструкции, — табулирование функции, то есть вычисление её значений в точках заданного интервала с постоянным шагом. Если функция задана несколькими выражениями, на каждом шаге цикла нужно определить, какое выражение использовать, и проверить, существует ли значение функции в данной точке.

В данной работе такая задача решается по варианту 7. Особенность варианта в том, что формат вывода результата зависит от поразрядного выражения, составленного из целых частей коэффициентов a , b и c . Программа, написанная самостоятельно, сравнивается с программой, которую по тому же заданию сгенерировала нейросеть DeepSeek.

1 ЦЕЛЬ И ХОД РАБОТЫ

1.1 Цель работы

Изучить приёмы использования операторов проверки условия и операторов цикла языка Си на примере табулирования функции, заданной несколькими выражениями, и сравнить собственную программу с программой, сгенерированной нейросетью.

1.2 Ход работы

Работа выполнялась в следующем порядке:

- 1) создана папка lab1, в ней подпапка step1 с двумя проектами: `tu` (собственная программа) и `ai` (программа нейросети);
- 2) разобрано условие варианта 7: определены ветви функции, точки, в которых функция не определена, и правило выбора формата вывода;
- 3) в проекте `tu` написана программа `tu.c`;
- 4) нейросети DeepSeek передано то же задание, полученный код сохранён в файле `ai/ai.c`;
- 5) обе программы скомпилированы компилятором GCC с флагами `-Wall -Wextra -std=c11`, предупреждений компилятор не выдал;
- 6) для собственной программы построена блок-схема алгоритма;
- 7) обе программы запущены на четырёх одинаковых наборах исходных данных;
- 8) при тестировании в первой версии собственной программы найдена ошибка на границе $x = 5$, программа исправлена;
- 9) исходные коды обеих программ проанализированы, результаты сведены в сравнительную таблицу, сделаны выводы.

2 ЗАДАНИЕ

Вычислить и вывести на экран в виде таблицы значения функции F на интервале от $X_{\text{нач.}}$ до $X_{\text{кон.}}$ с шагом dX . Индивидуальное задание соответствует варианту 7 и приведено на рисунке 2.1.

$$F = \begin{cases} -a \cdot x^2 - b & \text{при } x < 5 \text{ и } c \neq 0 \\ (x - a) / x & \text{при } x > 5 \text{ и } c = 0 \\ -x / c & \text{в остальных случаях} \end{cases}$$

Рисунок 2.1 – Индивидуальное задание, вариант 7

Здесь a , b , c — действительные числа. Функция F должна принимать действительное значение, если выражение $(A_{\text{ц}} \text{ ИЛИ } B_{\text{ц}}) \text{ МОД}2 (A_{\text{ц}} \text{ ИЛИ } C_{\text{ц}})$ не равно нулю, и целое значение в противном случае. Через $A_{\text{ц}}$, $B_{\text{ц}}$ и $C_{\text{ц}}$ обозначены целые части значений a , b , c ; операции ИЛИ и МОД2 (сложение по модулю 2) — поразрядные. Значения a , b , c , $X_{\text{нач.}}$, $X_{\text{кон.}}$, dX вводятся с клавиатуры.

2.1 Условия выбора ветвей и область определения

Функция задана тремя выражениями:

1) $F = -a \cdot x^2 - b$ при $x < 5$ и $c \neq 0$. В выражении нет деления, поэтому функция определена во всех точках, где работает эта ветвь;

2) $F = (x - a) / x$ при $x > 5$ и $c = 0$. В знаменателе стоит x , но ветвь работает только при $x > 5$, поэтому деления на ноль здесь быть не может;

3) $F = -x / c$ в остальных случаях: при $x = 5$ и любом c , при $x < 5$ и $c = 0$, при $x > 5$ и $c \neq 0$. Если при этом $c = 0$, получается деление на ноль. В таких точках функция не определена, и программа должна вывести сообщение вместо значения.

2.2 Поразрядное условие

Формат вывода определяется значением выражения $L = (A_{\text{ц}} \mid B_{\text{ц}}) \wedge (A_{\text{ц}} \mid C_{\text{ц}})$, где « $\mid$ » — поразрядное ИЛИ, а « $\wedge$ » — поразрядное сложение по модулю 2 (исключающее ИЛИ). Целые части берутся функцией `floor`, как в примере решения из методических указаний: $A_{\text{ц}} = (\text{long})\text{floor}(a)$. Для отрицательных нецелых чисел `floor` даёт ближайшее меньшее целое, например $\text{floor}(-0.5) = -1$.

Выражение L не зависит от x , поэтому оно вычисляется один раз до цикла. Если $L \neq 0$, значение функции выводится как действительное число с четырьмя знаками после запятой (формат `%.4f`). Если $L = 0$, значение выводится как целое, то есть округляется (формат `%.0f`).

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1 Код программы, написанной самостоятельно

Программа запрашивает коэффициенты a , b , c , границы интервала и шаг, проверяет введенные данные, до цикла вычисляет поразрядное условие и затем в цикле `for` выводит таблицу значений. Значение аргумента на каждом шаге вычисляется по номеру шага: $x = x_n + i \cdot dx$, поэтому погрешность сложения не накапливается. Сравнение x с границей 5 выполняется с допуском $EPS = 1e-9$.

Внутри цикла `for` расположены вложенные операторы `if — else if — else`: они выбирают нужное выражение и проверяют, определена ли функция в текущей точке. Результат проверки запоминается в переменной `is_defined`, после чего строка таблицы выводится в одном месте. Исходный код приведен ниже.

```
#include <stdio.h>
#include <math.h>

#ifdef _WIN32
#include <windows.h>
#endif

/* Допуск при сравнении вещественных чисел */
#define EPS 1e-9

int main(void)
{
    double a, b, c;
    double xn, xk, dx;
    double x, F;
    long i, n;
    int is_defined;

#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
#endif

    printf("=====\n");
    printf("    Лабораторная работа №1. Компьютерное программирование    \n");
    printf("    Тема: Операторы условия и циклов. Вариант 7                \n");
    printf("=====\n\n");

    printf("Введите коэффициенты a, b, c: ");
    if (scanf("%lf %lf %lf", &a, &b, &c) != 3)
    {
        printf("Ошибка: некорректный ввод коэффициентов a, b, c.\n");
        return 1;
    }

    printf("Введите начальное значение xn, конечное xk и шаг dx: ");
    if (scanf("%lf %lf %lf", &xn, &xk, &dx) != 3)
    {
        printf("Ошибка: некорректный ввод параметров интервала.\n");
        return 1;
    }

    if (dx <= 0.0 || xn > xk)
    {
        printf("Ошибка: некорректные параметры интервала "
               "(требуется dx > 0 и xn <= xk).\n");
        return 1;
    }

    /* Целые части коэффициентов (floor, как в методических указаниях) */
    long a_c = (long)floor(a);
    long b_c = (long)floor(b);
    long c_c = (long)floor(c);
```

```

/* Проверка битового условия: (Ац ИЛИ Вц) МОД2 (Ац ИЛИ Сц) */
long bitwise_cond = (a_c | b_c) ^ (a_c | c_c);
int print_as_real = (bitwise_cond != 0);

printf("\nАнализ исходных данных:\n");
printf("Целые части: Ац = %ld, Вц = %ld, Сц = %ld\n", a_c, b_c, c_c);
printf("Выражение (Ац | Вц) ^ (Ац | Сц) = %ld\n", bitwise_cond);
printf("Формат вывода F(x): %s\n\n",
       print_as_real ? "действительное число (условие != 0)"
                     : "целое число (условие == 0)");

printf("-----\n");
printf("      x              F(x)              \n");
printf("-----\n");

/* Количество шагов на интервале */
n = (long)floor((xk - xn) / dx + EPS);

/* Цикл табулирования: x считается по номеру шага,
   чтобы погрешность сложения не накапливалась */
for (i = 0; i <= n; i++)
{
    x = xn + i * dx;

    /* Ветвь 1: x < 5 и c != 0 */
    if (x < 5.0 - EPS && c != 0.0)
    {
        F = -a * x * x - b;
        is_defined = 1;
    }
    /* Ветвь 2: x > 5 и c == 0 */
    else if (x > 5.0 + EPS && c == 0.0)
    {
        F = (x - a) / x;
        is_defined = 1;
    }
    /* Ветвь 3: в остальных случаях */
    else
    {
        if (c != 0.0)
        {
            F = -x / c;
            is_defined = 1;
        }
        else
        {
            /* Деление на ноль при c == 0 */
            is_defined = 0;
        }
    }

    /* Вывод текущей строки таблицы */
    if (is_defined)
    {
        if (print_as_real)
            printf("%10.4f      %15.4f\n", x, F);
        else
            printf("%10.4f      %15.0f\n", x, F);
    }
    else
    {
        printf("%10.4f      Функция не определена\n", x);
    }
}

printf("-----\n");

return 0;
}

```

3.2 Блок-схема алгоритма

Блок-схема алгоритма собственной программы приведена на рисунке 3.1. Цикл изображён блоком модификации (шестиугольником), в котором записаны начальное значение счётчика, условие продолжения и шаг. После выполнения тела цикла управление возвращается в этот блок. Когда i становится больше n , цикл завершается и программа переходит к блоку «Конец». Буквой L на схеме обозначено значение поразрядного выражения (в программе — переменная `bitwise_cond`).

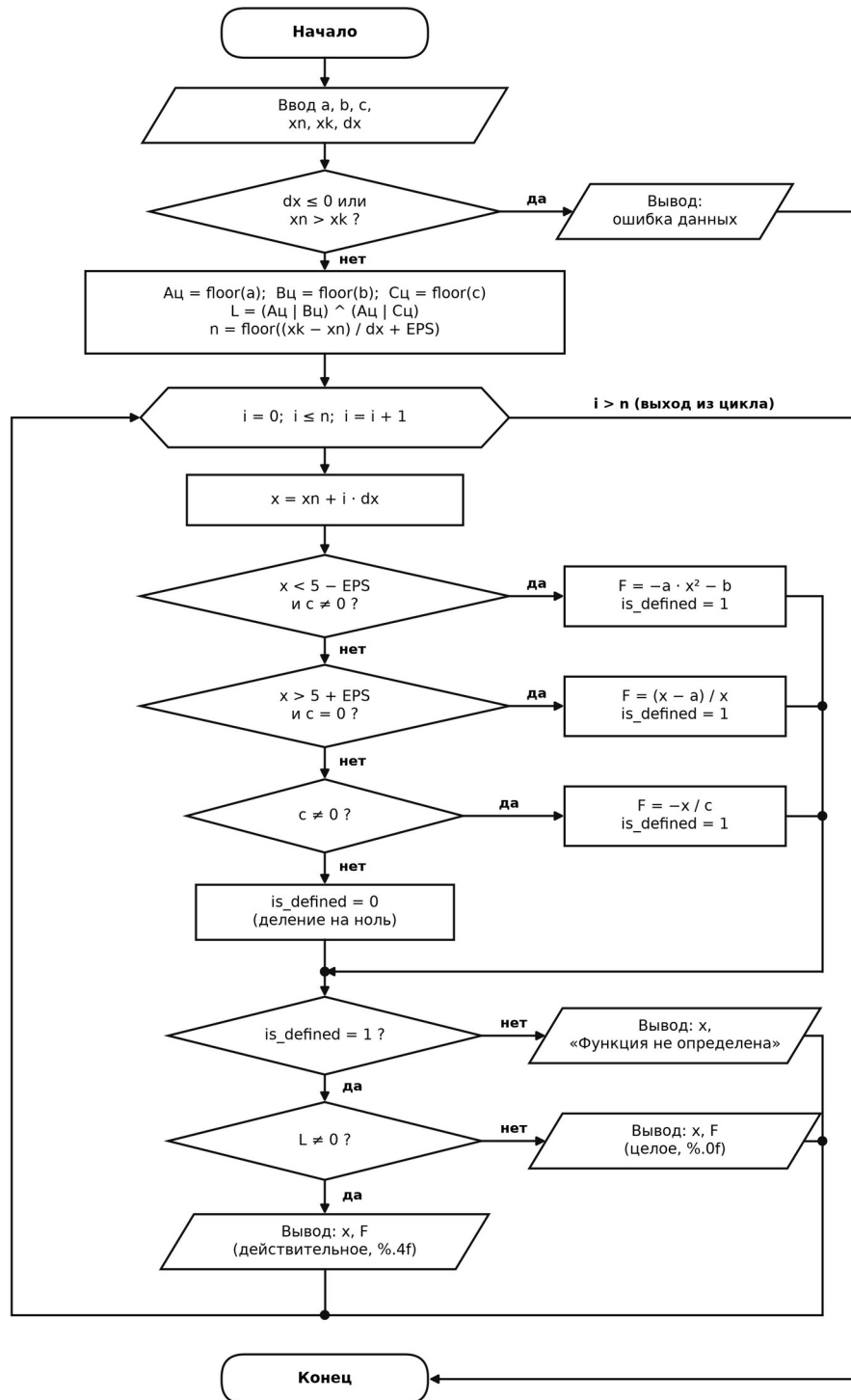


Рисунок 3.1 – Блок-схема алгоритма программы

3.3 Код программы, сгенерированный DeepSeek

Нейросети DeepSeek было передано следующее задание:

Напиши программу на языке C для табулирования кусочно-заданной функции:

$$F = \begin{cases} -a \cdot x^2 - b & \text{при } x < 5 \text{ и } c \neq 0 \\ (x - a) / x & \text{при } x > 5 \text{ и } c = 0 \\ -x / c & \text{в остальных случаях} \end{cases}$$

Функция F должна принимать действительное значение, если выражение (Ac ИЛИ Bc) MOD2 (Ac ИЛИ Cc) не равно нулю, и целое значение в противном случае. Через Ac, Bc и Cc обозначены целые части значений a, b, c, операции ИЛИ и MOD2 (сложение по модулю 2, XOR) – поразрядные. Входные данные: a, b, c, x_start, x_end, dx. Предусмотреть строгую проверку ОДЗ, защиту от деления на ноль, проверку входных параметров и вывод таблицы значений x и F(x).

Ниже приведён код, полученный от нейросети (проект ai, файл ai.c).

```
#include <stdio.h>
#include <math.h>
#include <stdbool.h>

#define EPS 1e-9

int main(void)
{
    double a, b, c;
    double x_start, x_end, dx;

    printf("Enter parameters a, b, c: ");
    if (scanf("%lf %lf %lf", &a, &b, &c) != 3) {
        printf("Error: Invalid input for a, b, c.\n");
        return 1;
    }

    printf("Enter x_start, x_end, dx: ");
    if (scanf("%lf %lf %lf", &x_start, &x_end, &dx) != 3) {
        printf("Error: Invalid input for interval.\n");
        return 1;
    }

    if (dx <= 0.0) {
        printf("Error: Step dx must be greater than zero.\n");
        return 1;
    }

    if (x_start > x_end) {
        printf("Error: x_start cannot be greater than x_end.\n");
        return 1;
    }

    /* Bitwise condition: (Ac | Bc) ^ (Ac | Cc) */
    long a_c = (long)trunc(a);
    long b_c = (long)trunc(b);
    long c_c = (long)trunc(c);

    bool is_real = (((a_c | b_c) ^ (a_c | c_c)) != 0);

    printf("\nBitwise condition value: %ld (Output: %s)\n",
        ((a_c | b_c) ^ (a_c | c_c)),
        is_real ? "Real" : "Integer");

    printf("\n%-12s %-18s\n", "x", "F(x)");
    printf("-----\n");

    for (double x = x_start; x <= x_end + EPS; x += dx) {
        double F = 0.0;
        bool undefined = false;

        /* Branch 1: x < 5 and c != 0 */
        if (x < 5.0 - EPS && fabs(c) > EPS) {
```

```

        F = -a * x * x - b;
    }
    /* Branch 2: x > 5 and c == 0 */
    else if (x > 5.0 + EPS && fabs(c) < EPS) {
        if (fabs(x) < EPS) {
            undefined = true;
        } else {
            F = (x - a) / x;
        }
    }
    /* Branch 3: Otherwise */
    else {
        if (fabs(c) < EPS) {
            undefined = true; /* Division by zero */
        } else {
            F = -x / c;
        }
    }

    if (undefined) {
        printf("%-12.4f %-18s\n", x, "undefined");
    } else {
        if (is_real) {
            printf("%-12.4f %-18.4f\n", x, F);
        } else {
            printf("%-12.4f %-18.0f\n", x, F);
        }
    }
}

printf("-----\n");

return 0;
}

```

Алгоритм в целом совпадает с собственным решением, но отличается способом получения целой части, организацией цикла и некоторыми проверками. Подробно различия разобраны в разделе 4.

4 СРАВНЕНИЕ ДВУХ ПРОГРАММ

4.1 Сравнительная таблица

Результаты сравнения двух программ по основным признакам приведены в таблице 4.1.

Таблица 4.1 – Сравнение собственной программы и программы DeepSeek

Признак	Моя программа (my.c)	Программа DeepSeek (ai.c)	Сравнение
Интерфейс	Сообщения на русском языке, в начале выводится заголовок работы.	Сообщения на английском языке, заголовка нет.	На вычисления не влияет. Русский интерфейс удобнее при проверке работы.
Переносимость	Под Windows через <code>#ifdef _WIN32</code> включается кодировка UTF-8 для консоли; в Linux этот код не компилируется.	Только стандартные заголовки <code>stdio.h</code> , <code>math.h</code> , <code>stdbool.h</code> .	Обе программы собираются и в Windows, и в Linux. Программе ИИ настройка кодировки не нужна: в ней нет русского текста.
Проверка ввода	Проверяется результат <code>scanf</code> . Условия <code>dx <= 0</code> и <code>xn > xk</code> проверяются вместе, сообщение общее.	Проверяется результат <code>scanf</code> . Условия <code>dx <= 0</code> и <code>x_start > x_end</code> проверяются отдельно, у каждой ошибки своё сообщение.	По надёжности одинаково. У ИИ сообщение точнее показывает, что именно введено неверно.
Целые части a, b, c	<code>(long)floor(a)</code> — как в примере из методических указаний.	<code>(long)trunc(a)</code> — дробная часть отбрасывается.	Для положительных чисел результат одинаковый. Для отрицательных нецелых — разный: <code>floor(-0.5) = -1</code> , <code>trunc(-0.5) = 0</code> . Из-за этого программы могут выбрать разный формат вывода (тест 4).
Поразрядное условие	Вычисляется один раз до цикла. Выводятся Ац, Вц, Сц и значение выражения.	Вычисляется один раз до цикла. Выводится только значение выражения.	Логика одинаковая. В моей программе расчёт проще проверить вручную.
Организация цикла	Цикл по целому счётчику <code>i</code> от 0 до <code>n</code> , на каждом шаге <code>x = xn + i * dx</code> .	К <code>x</code> прибавляется шаг: <code>x += dx</code> , условие цикла <code>x <= x_end + EPS</code> .	У ИИ погрешность сложения накапливается: при шаге 0.01 на интервале от -5 до 5 вместо 0.0000 выводится -0.0000. В моей программе этого нет.
Сравнение x с границей 5	<code>x < 5.0 - EPS</code> и <code>x > 5.0 + EPS</code> . В первой версии было <code>x < 5.0</code> и <code>x > 5.0</code> (см. тест 3).	<code>x < 5.0 - EPS</code> и <code>x > 5.0 + EPS</code> .	У ИИ сравнение с допуском было сразу. В моей программе оно появилось после найденной ошибки.

Признак	Моя программа (my.c)	Программа DeepSeek (ai.c)	Сравнение
Сравнение с с нулём	Точное: $c \neq 0.0$ и $c == 0.0$.	С допуском: $\text{fabs}(c) > \text{EPS}$ и $\text{fabs}(c) < \text{EPS}$.	Значение c вводится с клавиатуры, а не вычисляется, поэтому точное сравнение корректно. У ИИ очень малое c (например, $1e-10$) считается нулём, хотя по условию это случай $c \neq 0$.
Защита от деления на ноль	В третьей ветви при $c = 0$ выводится «Функция не определена».	В третьей ветви при $c = 0$ выводится <code>undefined</code> . Во второй ветви дополнительно проверяется $x = 0$.	Обе программы не делят на ноль. Проверка $x = 0$ у ИИ лишняя: вторая ветвь работает только при $x > 5$.
Вывод строки таблицы	Флаг <code>is_defined</code> , затем формат <code>%15.4f</code> или <code>%15.0f</code> . Выравнивание по правому краю.	Флаг <code>undefined</code> , затем формат <code>%-18.4f</code> или <code>%-18.0f</code> . Выравнивание по левому краю.	Структура одинаковая. При выравнивании по правому краю числа разного знака читать легче.
Результаты тестов	Тесты 1–3: значения верные. Тест 4: вывод целый.	Тесты 1–3: значения верные. Тест 4: вывод действительный.	Значения функции совпадают во всех тестах. Различается только формат вывода в тесте 4.

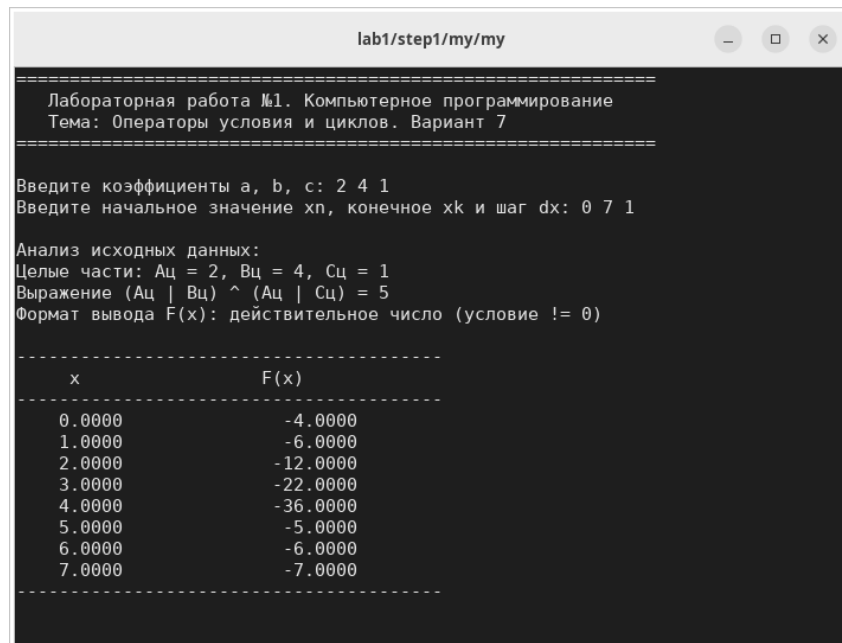
Из таблицы видно, что обе программы решают поставленную задачу, но отличаются в деталях. Программа DeepSeek с самого начала сравнивала x с границей через допуск, и именно сравнение с ней помогло найти ошибку в собственной программе. В то же время у нейросети целая часть берётся не так, как в методических указаниях, допуск без необходимости применён к параметру c , а значение x накапливает погрешность.

4.2 Результаты работы программ

Обе программы запускались на одинаковых исходных данных. Наборы данных подобраны так, чтобы сработали все три ветви функции, случай деления на ноль и оба формата вывода.

Тест 1. $a = 2$, $b = 4$, $c = 1$, x от 0 до 7 с шагом 1.

Целые части: $A_c = 2$, $B_c = 4$, $C_c = 1$. Поразрядное выражение: $(2 \mid 4) \wedge (2 \mid 1) = 6 \wedge 3 = 5$. Оно не равно нулю, поэтому значения выводятся как действительные числа. При x от 0 до 4 работает первая ветвь: $F = -2 \cdot x^2 - 4$. При $x = 5$ и далее ($c \neq 0$) работает третья ветвь: $F = -x / 1$. Результаты показаны на рисунках 4.1 и 4.2, значения совпадают.



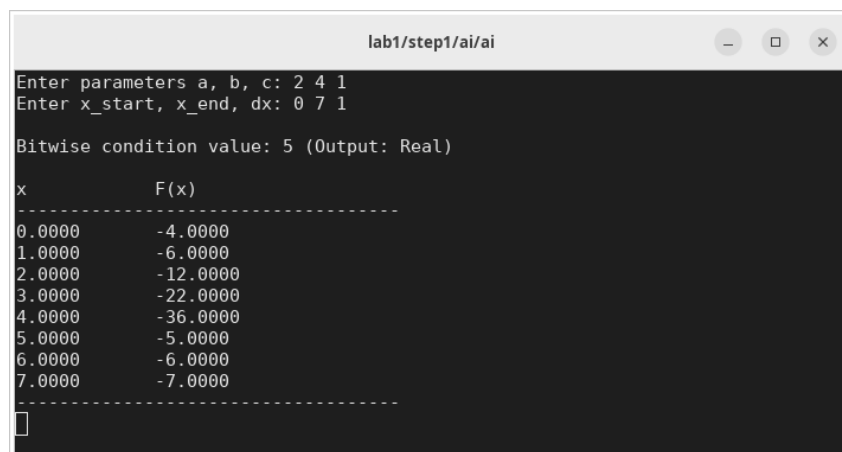
```
lab1/step1/my/my
=====
Лабораторная работа №1. Компьютерное программирование
Тема: Операторы условия и циклов. Вариант 7
=====

Введите коэффициенты a, b, c: 2 4 1
Введите начальное значение xp, конечное xk и шаг dx: 0 7 1

Анализ исходных данных:
Целые части: Ac = 2, Bc = 4, Cc = 1
Выражение (Ac | Bc) ^ (Ac | Cc) = 5
Формат вывода F(x): действительное число (условие != 0)

-----
      x          F(x)
-----
0.0000      -4.0000
1.0000      -6.0000
2.0000     -12.0000
3.0000     -22.0000
4.0000     -36.0000
5.0000      -5.0000
6.0000      -6.0000
7.0000      -7.0000
-----
```

Рисунок 4.1 – Тест 1, собственная программа



```
lab1/step1/ai/ai

Enter parameters a, b, c: 2 4 1
Enter x_start, x_end, dx: 0 7 1

Bitwise condition value: 5 (Output: Real)

x          F(x)
-----
0.0000      -4.0000
1.0000      -6.0000
2.0000     -12.0000
3.0000     -22.0000
4.0000     -36.0000
5.0000      -5.0000
6.0000      -6.0000
7.0000      -7.0000
-----

```

Рисунок 4.2 – Тест 1, программа DeepSeek

Тест 2. $a = 3.2$, $b = 1$, $c = 0$, x от 3 до 8 с шагом 1.

Целые части: $A_c = 3$, $B_c = 1$, $C_c = 0$. Поразрядное выражение: $(3 \mid 1) \wedge (3 \mid 0) = 3 \wedge 3 = 0$, поэтому значения выводятся как целые числа. При x от 3 до 5 работает третья ветвь, а так как $c = 0$, функция не определена. При x от 6 до 8 работает вторая ветвь: $F = (x - 3.2) / x$. Получаются значения 0.4667, 0.5429 и 0.6000, которые при выводе округляются до 0, 1 и 1. Результаты показаны на рисунках 4.3 и 4.4, значения совпадают.

```

lab1/step1/my/my

=====
Лабораторная работа №1. Компьютерное программирование
Тема: Операторы условия и циклов. Вариант 7
=====

Введите коэффициенты a, b, c: 3.2 1 0
Введите начальное значение x1, конечное xk и шаг dx: 3 8 1

Анализ исходных данных:
Целые части: Ац = 3, Вц = 1, Сц = 0
Выражение (Ац | Вц) ^ (Ац | Сц) = 0
Формат вывода F(x): целое число (условие == 0)

-----
x          F(x)
-----
3.0000    Функция не определена
4.0000    Функция не определена
5.0000    Функция не определена
6.0000      0
7.0000      1
8.0000      1
-----

```

Рисунок 4.3 – Тест 2, собственная программа

```

lab1/step1/ai/ai

Enter parameters a, b, c: 3.2 1 0
Enter x_start, x_end, dx: 3 8 1

Bitwise condition value: 0 (Output: Integer)

x          F(x)
-----
3.0000    undefined
4.0000    undefined
5.0000    undefined
6.0000      0
7.0000      1
8.0000      1
-----

```

Рисунок 4.4 – Тест 2, программа DeepSeek

Тест 3. $a = 1$, $b = 2$, $c = 1$, x от 4.8 до 5.2 с шагом 0.1 (граница $x = 5$ при дробном шаге).

По условию в точке $x = 5$ должна работать третья ветвь: $F = -5 / 1 = -5$. В первой версии собственной программы значение x получалось накоплением ($x += dx$) и сравнивалось с границей напрямую ($x < 5.0$). Число 0.1 в двоичном виде представляется неточно, поэтому вместо 5 получалось 4.999..., срабатывала первая ветвь, и программа выводила $F = -27.0000$ вместо -5.0000 . Программа DeepSeek на тех же данных выдавала правильное значение, так как сравнивает x с границей через допуск EPS.

После этого собственная программа была исправлена: x вычисляется по номеру шага, а сравнение с границей выполняется с допуском. На рисунках 4.5 и 4.6 показана работа исправленной программы и программы нейросети, значения совпадают.

```

lab1/step1/my/my

=====
Лабораторная работа №1. Компьютерное программирование
Тема: Операторы условия и циклов. Вариант 7
=====

Введите коэффициенты a, b, c: 1 2 1
Введите начальное значение x1, конечное xk и шаг dx: 4.8 5.2 0.1

Анализ исходных данных:
Целые части: Ац = 1, Вц = 2, Сц = 1
Выражение (Ац | Вц) ^ (Ац | Сц) = 2
Формат вывода F(x): действительное число (условие != 0)

-----
      x          F(x)
-----
4.8000      -25.0400
4.9000      -26.0100
5.0000      -5.0000
5.1000      -5.1000
5.2000      -5.2000
-----

```

Рисунок 4.5 – Тест 3, собственная программа (после исправления)

```

lab1/step1/ai/ai

Enter parameters a, b, c: 1 2 1
Enter x_start, x_end, dx: 4.8 5.2 0.1

Bitwise condition value: 2 (Output: Real)

x          F(x)
-----
4.8000      -25.0400
4.9000      -26.0100
5.0000      -5.0000
5.1000      -5.1000
5.2000      -5.2000
-----

```

Рисунок 4.6 – Тест 3, программа DeepSeek

Тест 4. $a = -0.5$, $b = 2.3$, $c = 1$, x от 3 до 6 с шагом 1 (отрицательный нецелый коэффициент).

В собственной программе $A_c = \text{floor}(-0.5) = -1$, $B_c = 2$, $C_c = 1$, и выражение равно $(-1 | 2) \wedge (-1 | 1) = (-1) \wedge (-1) = 0$, то есть вывод целый. В программе DeepSeek $A_c = \text{trunc}(-0.5) = 0$, выражение равно $(0 | 2) \wedge (0 | 1) = 2 \wedge 1 = 3$, и вывод действительный. Сами значения функции одинаковые (2.2; 5.7; -5; -6), но собственная программа выводит их округлёнными до целого, а программа нейросети — с четырьмя знаками после запятой (рисунки 4.7 и 4.8). Заданию соответствует первый вариант: в примере из методических указаний целая часть берётся функцией floor.

```

lab1/step1/my/my
=====
Лабораторная работа №1. Компьютерное программирование
Тема: Операторы условия и циклов. Вариант 7
=====
Введите коэффициенты a, b, c: -0.5 2.3 1
Введите начальное значение xp, конечное xk и шаг dx: 3 6 1

Анализ исходных данных:
Целые части: Ац = -1, Вц = 2, Сц = 1
Выражение (Ац | Вц) ^ (Ац | Сц) = 0
Формат вывода F(x): целое число (условие == 0)

-----
      x          F(x)
-----
3.0000          2
4.0000          6
5.0000         -5
6.0000         -6
-----

```

Рисунок 4.7 – Тест 4, собственная программа

```

lab1/step1/ai/ai
Enter parameters a, b, c: -0.5 2.3 1
Enter x_start, x_end, dx: 3 6 1

Bitwise condition value: 3 (Output: Real)

x          F(x)
-----
3.0000     2.2000
4.0000     5.7000
5.0000    -5.0000
6.0000    -6.0000
-----

```

Рисунок 4.8 – Тест 4, программа DeepSeek

Таким образом, в тестах 1–3 результаты программ полностью совпадают, а в тесте 4 программы по-разному выбирают формат вывода из-за разного способа получения целой части отрицательного числа.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены операторы проверки условия и оператор цикла `for` языка Си. С их помощью решена задача табулирования функции, заданной тремя выражениями.

Написанная программа вычисляет значения функции по варианту 7 на заданном интервале, проверяет введённые данные, не допускает деления на ноль и выбирает формат вывода по поразрядному условию. Для программы построена блок-схема алгоритма.

Сравнение с программой, сгенерированной нейросетью DeepSeek, оказалось полезным в обе стороны. Оно помогло найти ошибку в собственной программе: при дробном шаге точка $x = 5$ попадала не в ту ветвь. В коде нейросети тоже нашлись отступления от задания: целая часть отрицательных чисел берётся не так, как в методических указаниях, а очень малое значение ϵ считается нулём.

Основной вывод работы: вещественные числа, полученные в результате вычислений, нельзя сравнивать с границей напрямую, а сгенерированный код нельзя использовать без проверки — его нужно сверять с условием задания и тестировать на граничных значениях. Цель лабораторной работы достигнута.

ПРИЛОЖЕНИЕ. ССЫЛКА НА РЕПОЗИТОРИЙ

Исходные коды обеих программ (проекты `mu` и `ai`), блок-схема, снимки экрана и отчёт размещены в репозитории на платформе GitHub. Ссылка на репозиторий приведена ниже.

https://github.com/arseniistrokov/Lab1_CP_Variant7